

Phase-Adaptive Generation: Difficulty-Aware Semi-Autoregressive Decoding for Diffusion Language Models

Keshav Krishna Prajesh Kumar Siva Satvik Mandapati

New York University

Building LLM Reasoners — Spring 2026

{kk6081, pg2973, sm12779}@nyu.edu

Abstract

Phase-Adaptive Generation (PAG) is an original research project on difficulty-aware decoding for diffusion language models. We first built a baseline pipeline around LLaDA and Dream traces: collect token-level confidence signals (top-1 probability, top-2 probability, entropy, $\tau_{\text{stability}}$, and τ_{commit}), run change-point detection (CPD), derive segment tuples, and train a next-segment predictor. Through that process, we observed that CPD is useful for analysis but adds an avoidable intermediate step when the training target is block-level controls. Based on these observations, we now prioritize an AdaBlock-centered data path where traces directly provide block size and stabilization-step signals per block. This writeup documents both the attempted pipeline and the revised design decision, together with the current Transformer-based phase predictor and planned model ablations.

1 Introduction

Autoregressive (AR) language models generate one token at a time and therefore pay a latency cost that becomes significant on long reasoning traces. Diffusion-based LLMs (dLLMs) such as Dream 7B [Ye et al., 2025] and LLaDA [Nie et al., 2025] reduce this cost by generating in blocks through iterative denoising, but existing decoding policies are typically uniform: the same chunk size and refinement budget are applied regardless of whether the model is in an easy completion region or a difficult reasoning region.

Our project studies whether these generation traces contain reusable structure that can support better scheduling. The working hypothesis is that dLLM outputs move through difficulty phases: some spans stabilize quickly, while others remain uncertain and require more refinement. If those phases can be detected early enough, a scheduler can allocate compute more intelligently. In our current formulation, PAG aims to use phase predictions to adapt both chunk size and refinement intensity.

The writeup and codebase are organized to support that question in stages. We first collect traces from a baseline decoder, then segment the traces with change-point detection, then turn each segment into a tuple sequence, and finally predict the next segment tuple from previous ones. This staged setup is intentionally testable so that each step can yield intermediate evidence even before a full adaptive decoder is completed.

2 Related Work

Diffusion LLM decoding. Dream 7B [Ye et al., 2025] and LLaDA [Nie et al., 2025] show that diffusion-style decoding can be competitive with AR baselines on reasoning and planning tasks. Block Diffusion [Arriola et al., 2025] frames semi-autoregressive generation as a bridge between AR and diffusion decoding.

Adaptive block inference. AdaBlock-dLLM [Lu et al., 2025] is the closest prior idea to our project because it adapts block size based on confidence-like signals. Our project differs in two ways. First, we treat segmentation as an explicit learning problem over trace signals rather than only as a reactive confidence heuristic. Second, we predict a richer control tuple that can later drive both block size and refinement intensity.

Why this formulation matters. In the repository, the phase-analysis stage already exposes token-level traces, stabilizing features, and segment summaries. That makes the project a natural fit for a learned next-segment model: the model can operate on symbolic segment tuples rather than raw text, which keeps the method lightweight and interpretable.

3 Method

3.1 Pipeline Overview

The initial pipeline we implemented in the repository was: baseline decoding $\rightarrow$ trace collection $\rightarrow$ CPD phase segmentation $\rightarrow$ tuple construction $\rightarrow$ next-segment prediction $\rightarrow$ tuple-conditioned unmasking.

After running this workflow on Dream 7B and LLaDA traces, we revised the design direction. Our current preferred pipeline is: AdaBlock trace collection (block-native) $\rightarrow$ block tuple sequence construction $\rightarrow$ next-segment prediction $\rightarrow$ tuple-conditioned unmasking.

The key difference is that CPD is no longer required to recover block boundaries: AdaBlock traces already expose the block-level controls needed for training (block size and stabilization/refinement behavior), which reduces label-construction ambiguity and simplifies end-to-end training.

3.2 Step 1: Baseline Trace Collection

In the baseline trace-collection stage, we instrumented LLaDA and Dream generation to capture token-level trajectories over refinement steps. The tracked signals include top-1 probability, top-2 probability, entropy, and stabilization-oriented timing signals ($\tau_{\text{stability}}$, τ_{commit}). These trajectories were used as raw material for downstream phase analysis.

This design gave us rich diagnostics but also revealed a bottleneck: the model controls we ultimately need for scheduling are block-level values, while LLaDA/Dream traces are token-level. As a result, we had to add a CPD stage to infer breakpoints before producing block-size labels. The revised plan is to use AdaBlock traces directly, where block size and stabilization-step information are already present at the block granularity needed for training.

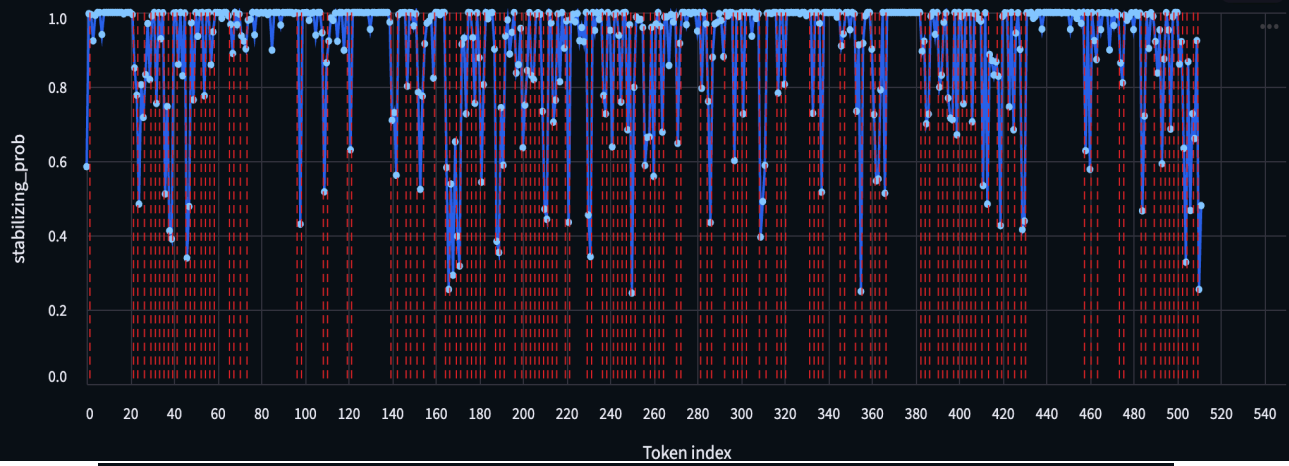
3.3 Step 2: Change-Point Detection and Segment Extraction

The second stage of the initial pipeline applies CPD to stabilization-oriented feature series. We used PELT with configurable cost (L2/normal), penalty, minimum segment length, and smoothing-window controls, and experimented across Dream 7B and LLaDA traces using scalar signals such as stabilizing entropy, margin, and top-1 probability.

Qualitatively, PELT produced useful structure: some prompts show clear regime shifts that align with intuitive reasoning transitions, while others are sensitive to smoothing and penalty settings. This made CPD valuable for understanding model behavior but less attractive as a permanent label-generation dependency for predictor training.

PAG Phase Segmentation Explorer	
Offline change-point analysis over curated diffusion trace files.	
Trace	Summary
Trace ID: prompt-005_entropy_stochastic_seed-0	Tokens
Backend: dream	512
Model: Dream-org/Dream-v0-Instruct-7B	Breakpoints
Prompt: Solve the equation $3(x - 4) + 5 = 2x + 17$ and explain each algebraic step clearly.	138
Tags: math, algebra, step_by_step	Segments
Trace profile: entropy (temperature=0.0, alg_temp=0.1)	139
Generation temperature: 0.0	Feature Std
Refinement alg_temp: 0.1	0.1731
Seed: 0	Feature
Preferred training profile: deterministic token generation with small refinement-order randomness.	stabilizin

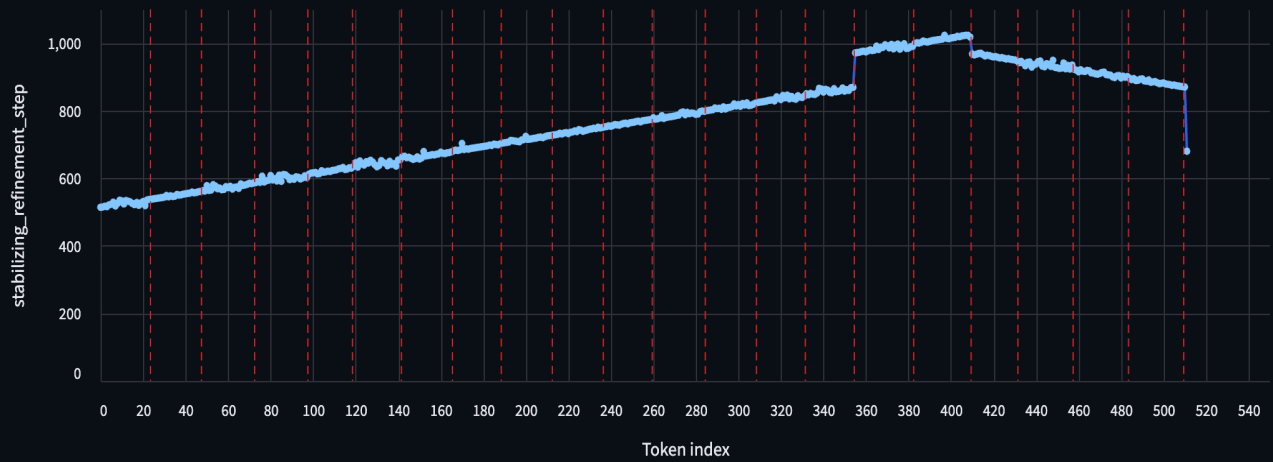
Feature vs token index



Segments ⇌

	segment	span	length	text	mean	std	min	max
0	0	[0, 2)	2	To solve	0.7909	0.2059	0.585	0.9967
1	1	[2, 22)	20	the equation $3(x - 4) + 5 = 2x + 17$	0.992	0.0203	0.9237	1
2	2	[22, 24)	2	, we	0.814	0.0367	0.7773	0.8507
3	3	[24, 27)	3	will follow these	0.669	0.1354	0.4843	0.8054
4	4	[27, 30)	3	steps: 1	0.8741	0.0681	0.8195	0.9701
5	5	[30, 32)	2	. Distrib	0.9954	0.0045	0.9909	1
6	6	[32, 34)	2	ute the	0.8772	0.1223	0.7549	0.9995
7	7	[34, 36)	2	3	0.9642	0.0351	0.9291	0.9992
8	8	[36, 38)	2	into the	0.6291	0.1178	0.5113	0.7469
9	9	[38, 40)	2	parentheses on	0.4013	0.0111	0.3902	0.4124

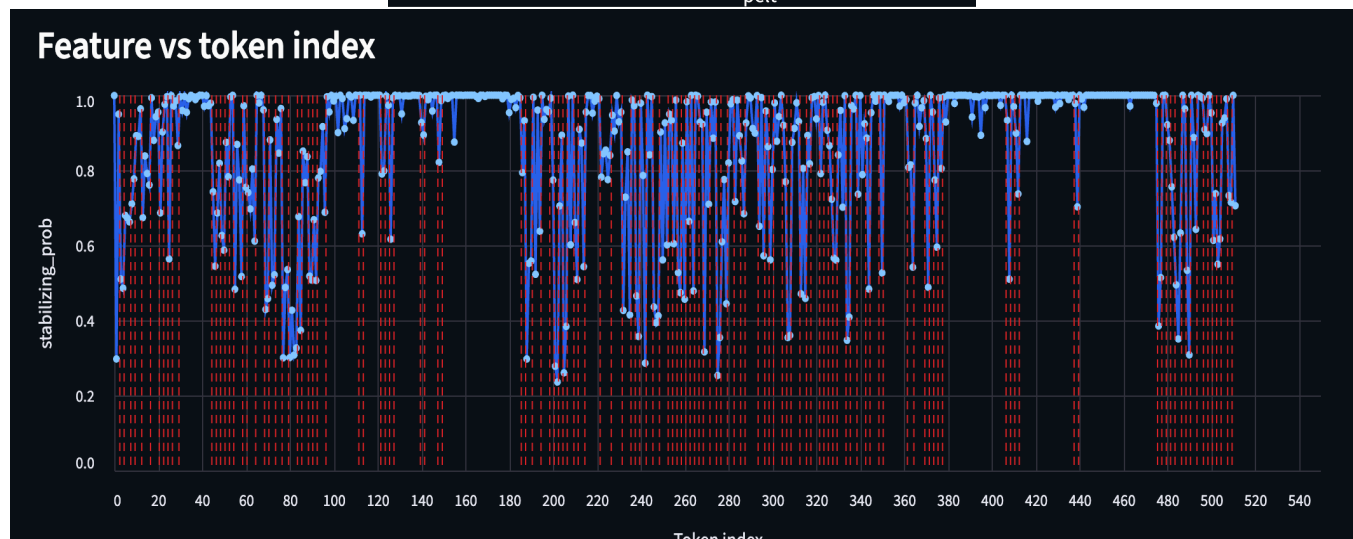
Feature vs token index



	segment	span	length	text	mean	std	min	ma
12	12	[285, 309]	24	involves moving all the x terms to one side of the equation. By subtracting 2x from both sides, we	811.5	6.9222	800	£
13	13	[309, 332]	23	get $x - 7 = 17$. 4. Add 7 to both sides: This step involves	835	6.6332	824	£
14	14	[332, 355]	23	isolating the variable x. By adding 7 to both sides, we get $x = 24$.	858	6.6332	847	£
15	15	[355, 383]	28	By following these steps, we have solved the equation $3(x - 4) + 5 = 2x + 17$	984.5	8.0777	971	£
16	16	[383, 410]	27	and found that $x = 24$. This means that when x is 24, both sides of the equation will be equal	1.8889	7.6174	999	10
17	17	[410, 432]	22	. To check our solution, we can substitute $x = 24$ back into the original equation: 3	19.4545	6.4225	948	£
18	18	[432, 458]	26	$(24 - 4) + 5 = 2(24) + 17$ 3/20	15.5385	7.5663	923	£
19	19	[458, 484]	26	$) + 5 = 48 + 17$ 60 + 5 = 48 + 17	909.5	7.5	897	£
20	20	[484, 510]	26	$65 = 65$ As you can see, both sides of the equation are equal, confirming that our solution is	883.5	7.5	871	£
21	21	[510, 512]	2	correct.	775	95	680	£

Inference: For a prompt in algebra, the stabilizing probability signal shows good block structure, and PELT surfaces breakpoints that align with intuitive phase transitions. But looking at stabilizing refinement steps signal, the increment is monotonic and hard to predict exact breakpoints. And these breakpoints are sensitive to the penalty parameter as well, which motivates our shift toward using AdaBlock-native traces where block boundaries and stabilization signals are directly available without CPD hyperparameters.

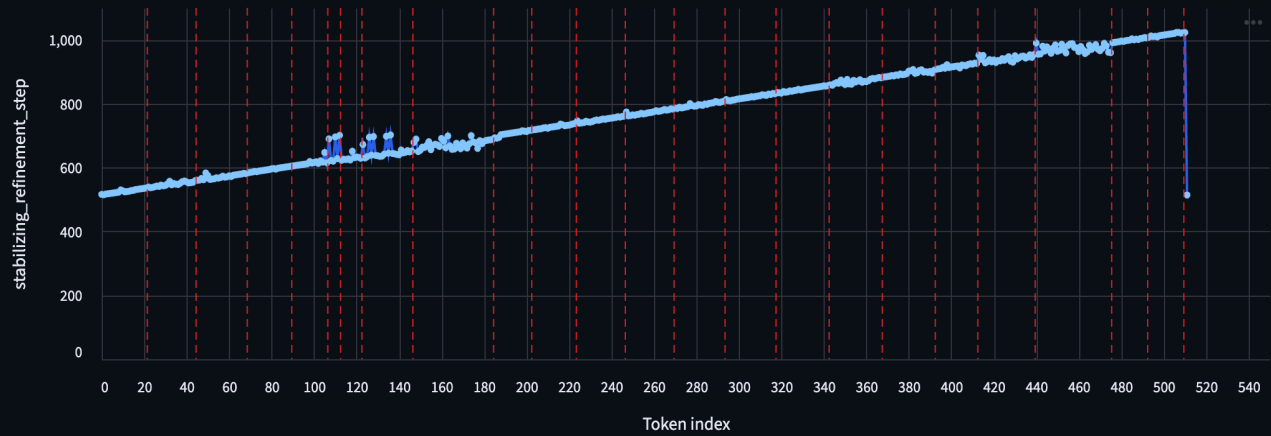
Trace	Summary
Trace ID: prompt-011_entropy_stochastic_seed-0	Tokens 512
Backend: dream	Breakpoints 129
Model: Dream-org/Dream-v0-Instruct-7B	Segments 130
Prompt: Implement binary search in Python and explain the loop invariants that make it correct.	Feature Std 0.1981
Tags: code, python, correctness	Feature stabilizing_prob
Trace profile: entropy (temperature=0.0, alg_temp=0.1)	Detector pelt
Generation temperature: 0.0	
Refinement alg_temp: 0.1	
Seed: 0	
Preferred training profile: deterministic token generation with small refinement-order randomness.	



Segments

	segment	span	length	text	mean	std	min	max
33	33	[114, 122]	8	0, len(arr) -	0.9993	0.0016	0.9952	1
34	34	[122, 124]	2	1	0.7943	0.0042	0.7901	0.7985
35	35	[124, 126]	2	while	0.9856	0.0124	0.9732	0.998
36	36	[126, 128]	2	left <=	0.8048	0.1885	0.6163	0.9933
37	37	[128, 140]	12	right: mid = (left + right) //	0.9949	0.0135	0.9501	1
38	38	[140, 142]	2	2	0.9106	0.0167	0.8939	0.9273
39	39	[142, 148]	6	if arr[mid] ==	0.9904	0.0151	0.9578	1
40	40	[148, 150]	2	target:	0.9035	0.0821	0.8213	0.9856
41	41	[150, 186]	36	return mid elif arr[mid] < target: left = mid + 1 else: right = r	0.9925	0.0219	0.8752	1
42	42	[186, 188]	2	```	0.863	0.0692	0.7938	0.9322
43	43	[188, 191]	3	The loop in	0.4692	0.1216	0.2973	0.5584

Feature vs token index



Segments

	segment	span	length	text	mean	std	min	max
0	0	[0, 22]	22	Binary search is an efficient algorithm for finding an element in a sorted array. It work	525.5	6.3443	515	536
1	1	[22, 45]	23	interval in half. If the value of the search key is less than the item in the middle of the	548	6.6332	537	559
2	2	[45, 69]	24	the algorithm repeats the process on the lower half. If the key is greater, the algorithm	571.5	6.9222	560	583
3	3	[69, 90]	21	Repeatedly dividing the interval in half, the algorithm has found the value or the inter	594	6.0553	584	604
4	4	[90, 107]	17	Here is a Python implementation of binary search: ```python def binary_search(arr,	614.5882	9.3057	605	647
5	5	[107, 113]	6	target): left, right	659.5	36.3857	620	701
6	6	[113, 123]	10	= 0, len(arr) - 1	630.1	7.687	623	651
7	7	[123, 147]	24	while left <= right: mid = (left + right) // 2 if arr[mid]	652.9583	21.8336	630	702
8	8	[147, 185]	38	== target: return mid elif arr[mid] < target: left = mid + 1 else:	673.3421	12.4547	650	700
9	9	[185, 203]	18	``` The loop invariants that make binary search correct are: 1. The array	707.1667	7.4703	692	717

Inference: For a prompt in coding, the stabilizing probability signal shows some structure (especially for the code tokens) but is noisier, and PELT breakpoints are less stable across penalty settings. Similar monotonic behavior is observed in the refinement steps signal as compared to the algebraic case. This again supports our decision to use

AdaBlock traces directly for training, where block-level controls are explicit and CPD is not required to recover them.

Because our predictor target is block-level scheduling, we ultimately decided to use AdaBlock traces for training labels, where block size and stabilization-step signals are directly available without running CPD to recover break-points.

3.4 Step 3: Tuple Sequences and Next-Segment Prediction

The third stage represents each block/segment as an ordered control tuple and learns next-tuple prediction from history. In the current implementation (`phase_predict`), a tuple contains at least (`block_size`, `refinement_steps`), with schema support for adding additional control dimensions later.

Our current predictor is a compact Transformer encoder trained as a regression model over normalized tuple sequences. Given a context window, the model predicts the next tuple and maps outputs back to integer scheduler controls. Compared with fixed-order heuristics, this architecture can model longer-range dependencies across blocks while remaining lightweight.

We treat this as a strong baseline rather than a final choice. Planned ablations include alternative sequence models (for example MLP/linear baselines, recurrent variants, and other Transformer sizes) to identify the best quality-latency trade-off for control prediction.

3.5 Step 4: Tuple-Conditioned Diffusion Unmasking

The fourth stage consumes the predicted tuple as a scheduling command for the next unmasking action. If the predicted tuple is (b, τ) , block size b controls how many positions are unmasked in the next decode block, and τ controls stabilization/refinement intensity before committing.

In the revised pipeline, this loop is designed around AdaBlock-native traces and controls: train on observed block sequences, predict the next block tuple, apply tuple-conditioned unmasking, append the realized block outcome, and repeat. This keeps the training target and inference control space aligned and removes the CPD-derived boundary recovery step from the production training path.

4 Implementation Status

The repository currently includes both the analysis-first pipeline and the revised training direction. The `phase_cpd` package supports trace exploration, stabilization-feature extraction, and CPD experiments (including PELT controls and feature choices). The `phase_predict` package implements tuple extraction, dataset construction, a Transformer next-segment predictor, training/inference wrappers, and checkpointing.

Practically, we view the CPD path as an exploratory analysis tool that helped us interpret phase structure in Dream/LLaDA traces, while the AdaBlock path is the intended data source for final predictor training because it provides block-size and stabilization-step signals directly. Remaining work is to complete the AdaBlock-integrated end-to-end loop and compare predictor variants under a common evaluation protocol.

5 Preliminary Analysis and Limitations

Our preliminary takeaway from CPD experiments is directional: PELT is effective for surfacing phase-like structure and for diagnosing where confidence dynamics shift, but it also introduces hyperparameter-sensitive boundary decisions when used as a mandatory label-construction step. That observation motivated the design change toward AdaBlock-native traces for predictor training.

The main current limitation is evaluation depth: we still need systematic held-out comparisons across predictor families and direct downstream measurements of quality, latency, and compute efficiency. A second limitation is that the present tuple schema is intentionally compact (block size and refinement/stabilization behavior), so richer controls may be needed if future experiments show underfitting of complex phase transitions.

6 Conclusion

PAG frames adaptive diffusion decoding as a next-control prediction problem over block/segment tuples. Our project progress includes an end-to-end attempt built on LLaDA/Dream traces plus PELT-based segmentation, and a revised pipeline decision informed by those results: train on AdaBlock-native traces where block size and stabilization-step controls are directly available. This revision simplifies label construction, aligns training and inference controls, and preserves CPD as a valuable analysis tool rather than a required production dependency.

References

- J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong. Dream 7B: Diffusion large language models. *arXiv:2508.15487*, 2025.
- S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li. Large language diffusion models. *arXiv:2502.09992*, 2025.
- G. Lu, H. M. Chen, Y. Karashima, Z. Wang, D. Fujiki, and H. Fan. AdaBlock-dLLM: Semantic-aware diffusion LLM inference via adaptive block size. *ICLR*, 2026.
- M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *ICLR (Oral)*, 2025.
- C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *ICLR*, 2026.
- S. Gong, S. Agarwal, Y. Zhang, J. Ye, L. Zheng, M. Li, C. An, P. Zhao, W. Bi, J. Han, H. Peng, and L. Kong. Scaling diffusion language models via adaptation from autoregressive models. *ICLR*, 2025.